

Programowanie obiektowe i inżynieria oprogramowania

Lab AG-9:

- dziedziczenie;
- funkcja wirtualna.

Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
AGH University of Science and Technology

Prowadzący: dr inż. Jakub Bryła
2025

Napiszemy różne klasy dla osobnika, pozwalające na wykorzystanie algorytmu genetycznego dla różnych funkcji celu oraz/lub różnego zakresu wartości przyjmowanych przez poszczególne geny w genotypie osobnika.

Ograniczymy się do osobników zawierających w genotypie dokładnie **dwa geny**.

Utworzona do tej pory klasa TCandidate będzie naszą klasą podstawową, „rodzicem” dla pozostałych wersji klasy osobnika.

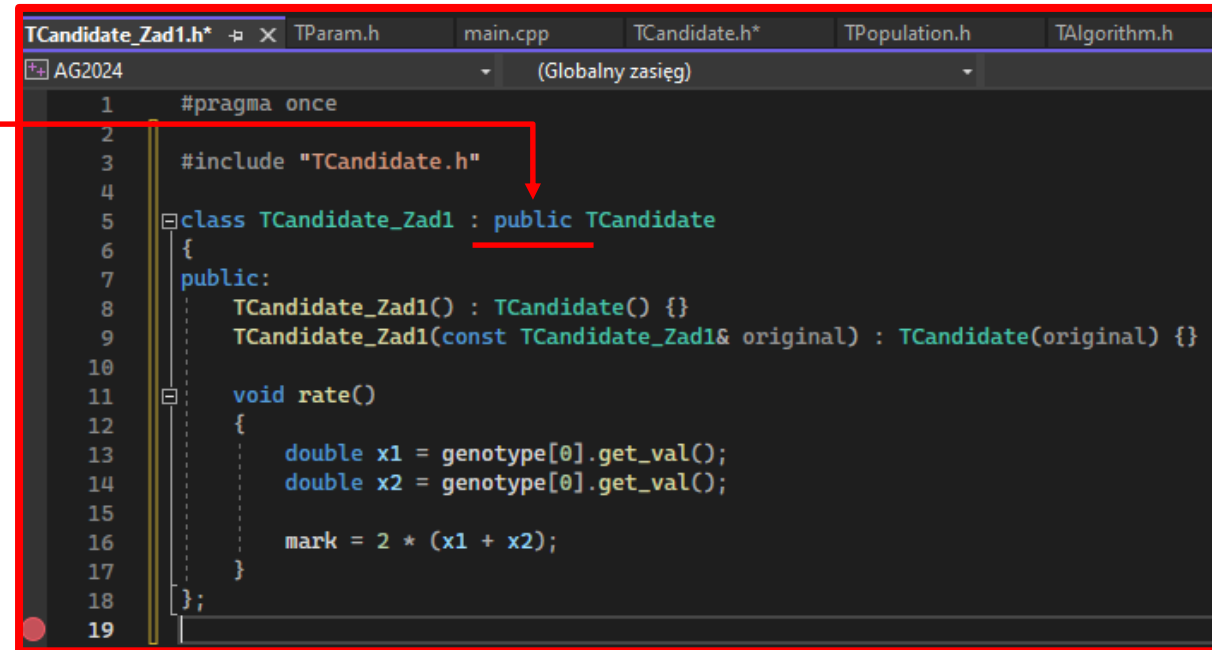
Aby klasy pochodne mogły dziedziczyć parametry/funkcje klasy podstawowej muszą być one oznaczone atrybutem *protected* – będą one widoczne w klasach dziedziczących, dla pozostałego kodu będą niewidoczne (jak w przypadku *private*).

```
n.h  main.cpp  TCandidate.h*  TPopulation.h  TAlgorithm.h
2024  (Globalny zasięg)
1  #pragma once
2  #include "TParam.h"
3
4  #define GENS_COUNT 2
5
6  class TCandidate
7  {
8  protected:
9      TParam genotype[GENS_COUNT] =
10     {
11         TParam("x1", 0, 100, 1),
12         TParam("x2", 0, 10, 1)
13     };
14
15     double mark;
16
17 public:
18     TCandidate();
19     TCandidate(const TCandidate &original);
20
21     double get_mark() const { return mark; };
22     void rate();
23
24     void info();
25
26 protected:
27     void rand_gens_val();
28     double get_gen_val(int gen_id) const { return genotype[gen_id].get_val(); }
29 };
30
```

Tworzymy klasę kandydata, która dziedziczy po klasie bazowej *TCandidate*. Posiada one te same parametry i funkcje składowe co klasa podstawowa, zmieniamy jedynie postać funkcji celu.

Po dwukropku podajemy klasę podstawową oraz atrybut *public* / *protected* / *private*:

- Dla *public*: parametry/funkcje składowe public w klasie podstawowej są publiczne w klasie dziedziczącej;
- Dla *protected*: parametry/funkcje składowe public w klasie podstawowej są chronione w klasie dziedziczącej;
- Dla *private*: parametry/funkcje składowe *public* i *protected* w klasie podstawowej są prywatne w klasie dziedziczącej.

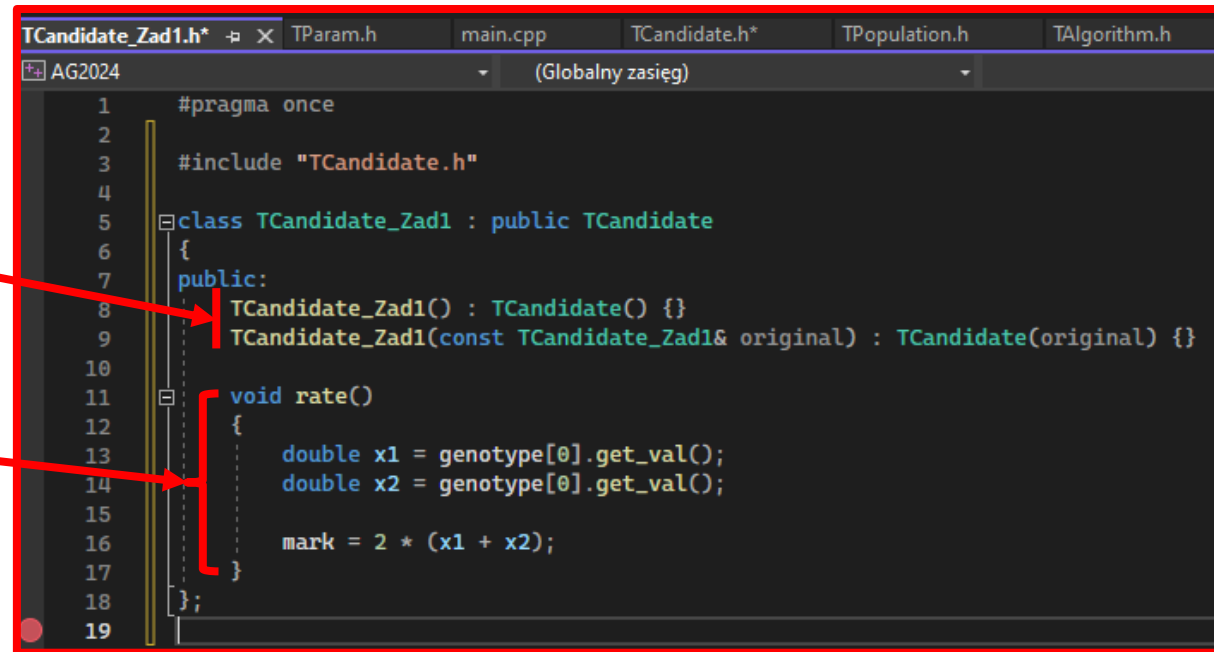


```
1  #pragma once
2
3  #include "TCandidate.h"
4
5  class TCandidate_Zad1 : public TCandidate
6  {
7  public:
8      TCandidate_Zad1() : TCandidate() {}
9      TCandidate_Zad1(const TCandidate_Zad1& original) : TCandidate(original) {}
10
11  void rate()
12  {
13      double x1 = genotype[0].get_val();
14      double x2 = genotype[0].get_val();
15
16      mark = 2 * (x1 + x2);
17  }
18  };
19
```

Tworzymy klasę kandydata, która dziedziczy po klasie *TCandidate*. Posiada one te same parametry i funkcje składowe co klasa podstawowa, zmieniamy jedynie postać funkcji celu.

Konstruktory nie są dziedziczone! Jednak dzięki konstruktorom delegującym można oszczędzić sobie trudu przepisywania kodu.

Nowa postać funkcji celu osobnika.



```
1  #pragma once
2
3  #include "TCandidate.h"
4
5  class TCandidate_Zad1 : public TCandidate
6  {
7  public:
8      TCandidate_Zad1() : TCandidate() {}
9      TCandidate_Zad1(const TCandidate_Zad1& original) : TCandidate(original) {}
10
11      void rate()
12      {
13          double x1 = genotype[0].get_val();
14          double x2 = genotype[0].get_val();
15
16          mark = 2 * (x1 + x2);
17      }
18  };
19
```

Dzięki dziedziczeniu utworzenie nowej klasy osobnika, różniącego się jedynie funkcją celu, nie wymaga dużego nakładu pracy i kodu – wszystkie zmiany wprowadzamy do pliku nagłówkowego. Tym samym nie ma potrzeby tworzenia pliku źródłowego.

```
main.cpp  TParam.h  TCandidate.h  TCan
AG2024
1  #include <iostream>
2  #include <cstdlib> // srand
3  #include <time.h>
4
5  #include "TCandidate.h"
6  #include "TCandidate_Zad1.h"
7
8  using namespace std;
9
10
11 int main()
12 {
13     srand(time(0));
14
15     TCandidate os_org{};
16     os_org.rate();
17     os_org.info();
18
19     TCandidate_Zad1 os_zad1{};
20     os_zad1.rate();
21     os_zad1.info();
22
23     std::cout << "\n\n\n";
24     return 0;
25 }
```

Konsola debugowania programu Microsoft Visual Studio

```
=====
== gens count: 2
== "x1" value: 54
== "x2" value: 6
== rate: 2922
=====

void TCandidate::rate()
{
    double x1 = genotype[0].get_val();
    double x2 = genotype[1].get_val();

    mark = pow(x1, 2) + x2;
}

=====
== gens count: 2
== "x1" value: 29
== "x2" value: 5
== rate: 68
=====

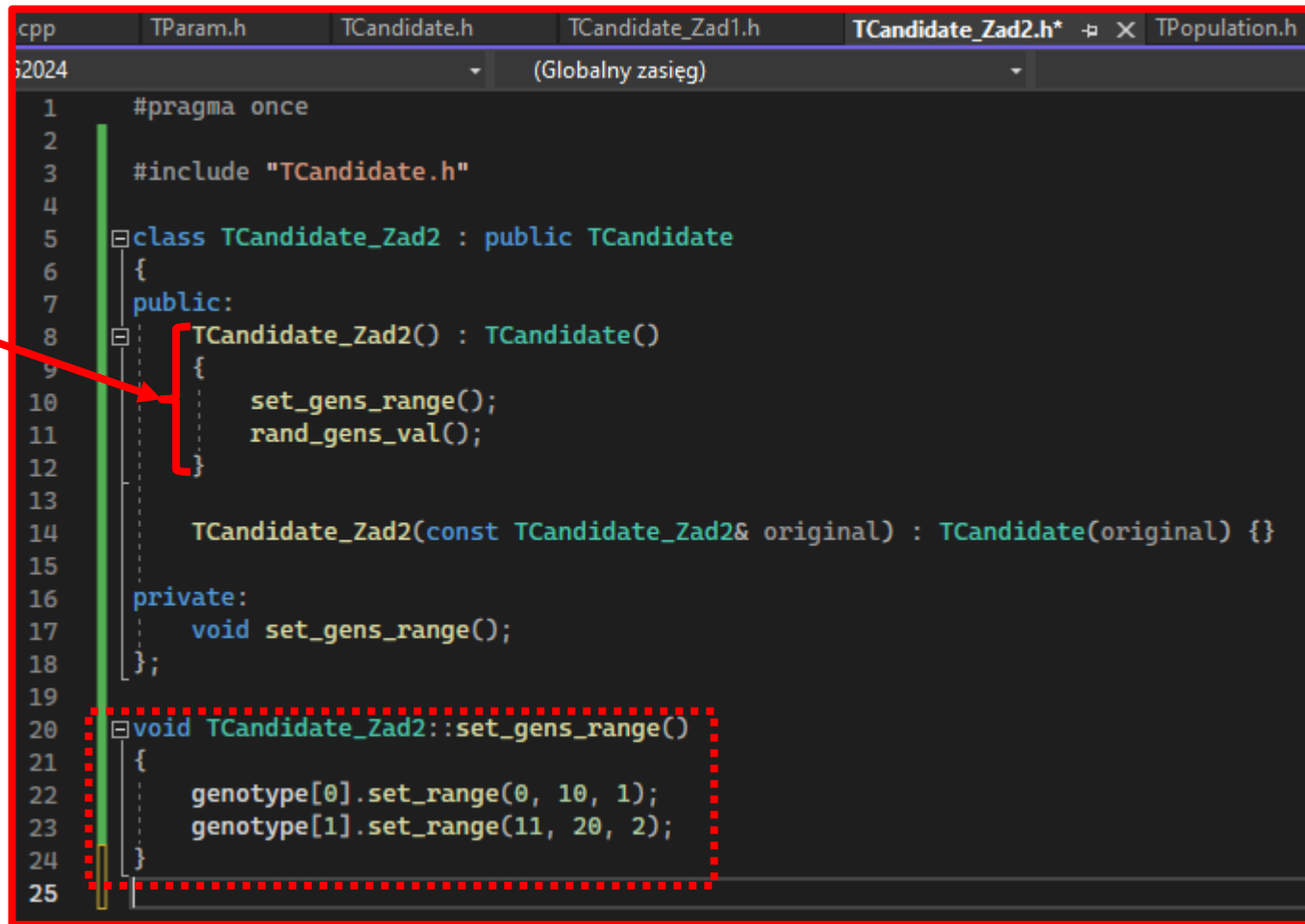
void rate()
{
    double x1 = genotype[0].get_val();
    double x2 = genotype[1].get_val();

    mark = 2 * (x1 + x2);
}
```

Tworzymy jeszcze jedną klasę osobnika, dziedziczącą po klasie *TCandidate*. Tym razem zmianie podlega zakres wartości przyjmowanych przez poszczególne geny genotypu osobnika.

Do utworzenia obiektu wywołujemy konstruktor klasy podstawowej. Musimy jednak zaktualizować zakres wartości przyjmowanych przez gen. W tym celu tworzymy funkcję pomocniczą (linie 20-24) wywoływaną w ciele konstruktora (linia 10).

Następnie, dla nowego zakresu, losujemy wartości genów (linia 11) – funkcja ta została zdefiniowana w klasie podstawowej.



```
1  #pragma once
2
3  #include "TCandidate.h"
4
5  class TCandidate_Zad2 : public TCandidate
6  {
7  public:
8      TCandidate_Zad2() : TCandidate()
9      {
10         set_gens_range();
11         rand_gens_val();
12     }
13
14     TCandidate_Zad2(const TCandidate_Zad2& original) : TCandidate(original) {}
15
16 private:
17     void set_gens_range();
18 };
19
20 void TCandidate_Zad2::set_gens_range()
21 {
22     genotype[0].set_range(0, 10, 1);
23     genotype[1].set_range(11, 20, 2);
24 }
25
```


TcandidateZad2.h - TEST

```
main.cpp  TParam.h  TCandidate.h  TCandidate_Zad1.h
AG2024 (Globalny zasięg)
1 #include <iostream>
2 #include <cstdlib> // srand
3 #include <time.h>
4
5 #include "TCandidate_Zad2.h"
6
7 using namespace std;
8
9
10 int main()
11 {
12     srand(time(0));
13
14     TCandidate_Zad2 os_zad2{};
15     os_zad2.rate();
16     os_zad2.info();
17
18     TCandidate_Zad2 os_zad2_copy{os_zad2};
19     os_zad2_copy.rate();
20     os_zad2_copy.info();
21
22     std::cout << "\n\n\n";
23     return 0;
24 }
```

```
Konsola debugowania programu Microsoft Visual Studio
=====
== gens count: 2
== "x1" value: 4
== "x2" value: 17
== rate: 33
=====

void TCandidate_Zad2::set_gens_range()
{
    genotype[0].set_range(0, 10, 1);
    genotype[1].set_range(11, 20, 2);
}

=====

class TCandidate
{
protected:
    TParam genotype[GENS_COUNT] =
    {
        TParam("x1", 0, 100, 1),
        TParam("x2", 0, 10, 1)
    };
}

=====
== gens count: 2
== "x1" value: 4
== "x2" value: 10
== rate: 26
=====
```

Obiekt `os_zad2_copy` przyjmuje zakres wartości genów przyjętych domyślnie w klasie `TCandidate`.

Wymagane są zmiany dla prawidłowego działania konstruktora kopiującego (linia 18) w przypadku zmiany zakresu wartości przyjmowanych przez geny osobnika.

[Nowy fragment kodu]

Domyślnie nowo tworzony obiekt przyjmuje zakres wartości genów określony w pliku nagłówkowym klasy *TCandidate*. Nam jednak zależy, aby zakres był taki sam jak dla wzorca – obiektu *kopia*. Tym samym kopiujemy wartości odpowiednich parametrów (linie 20-22) i aktualizujemy zakres dla nowego obiektu (linia 23).

```
14 TCandidate::TCandidate(const TCandidate &original)
15 {
16     mark = original.get_mark();
17
18     for (int i = 0; i < GENS_COUNT; i++)
19     {
20         double x_start = original.genotype[i].get_x_start();
21         double x_end = original.genotype[i].get_x_end();
22         double dx = original.genotype[i].get_dx();
23         genotype[i].set_range(x_start, x_end, dx);
24
25         double val = original.get_gen_val(i);
26         genotype[i].set_val(val);
27     }
28 }
```

TcandidateZad2.h - TEST

```
main.cpp x TParam.h TCandidate.h TCandidate_Zad1.h
AG2024 (Globalny zasięg)
1 #include <iostream>
2 #include <cstdlib> // srand
3 #include <time.h>
4
5 #include "TCandidate_Zad2.h"
6
7 using namespace std;
8
9
10 int main()
11 {
12     srand(time(0));
13
14     TCandidate_Zad2 os_zad2{};
15     os_zad2.rate();
16     os_zad2.info();
17
18     TCandidate_Zad2 os_zad2_copy{os_zad2};
19     os_zad2_copy.rate();
20     os_zad2_copy.info();
21
22     std::cout << "\n\n\n";
23     return 0;
24 }
```



```
Konsola debugowania programu Microsoft Visual Studio

=====
== gens count: 2
== "x1" value: 9
== "x2" value: 13
== rate: 94
=====

void TCandidate_Zad2::set_gens_range()
{
    genotype[0].set_range(0, 10, 1);
    genotype[1].set_range(11, 20, 2);
}

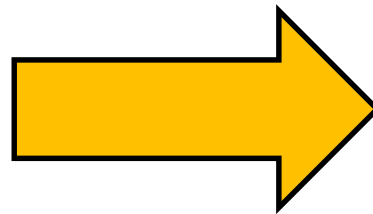
=====
== gens count: 2
== "x1" value: 9
== "x2" value: 13
== rate: 94
=====

void TCandidate_Zad2::set_gens_range()
{
    genotype[0].set_range(0, 10, 1);
    genotype[1].set_range(11, 20, 2);
}
```

```
class TCandidate
{
protected:
    TParam genotype[GENS_COUNT] =
    {
        TParam("x1", 0, 100, 1),
        TParam("x2", 0, 10, 1)
    };
};
```

Funkcja wirtualna

```
main.cpp x TParam.h TCandidate.h TCandidate_Zad1.h
AG2024 (Globalny zasięg)
1 #include <iostream>
2 #include <cstdlib> // srand
3 #include <time.h>
4
5 #include "TCandidate.h"
6 #include "TCandidate_Zad1.h"
7 #include "TCandidate_Zad2.h"
8
9 using namespace std;
10
11
12 int main()
13 {
14     srand(time(0));
15
16     TCandidate* os = new TCandidate_Zad1{};
17     os->rate();
18     os->info();
19     delete os;
20
21     std::cout << "\n\n\n";
22     return 0;
23 }
```



```
Konsola debugowania programu
=====
==
== gens count: 2
== "x1" value: 9
== "x2" value: 3
==
== rate: 84
=====
```

```
void rate()
{
    double x1 = genotype[0].get_val();
    double x2 = genotype[1].get_val();

    mark = 2 * (x1 + x2);
}
```

Dzięki dziedziczeniu można wskazywać na klasy pochodne za pomocą wskaźnika do klasy podstawowej (linia 16)!

Niemniej w tym przypadku zostały wywołane funkcje klasy podstawowej, podczas gdy chcieliśmy obliczyć wynik osobnika dla zmodyfikowanej funkcji celu – wywołać funkcję *rate()* zdefiniowaną w klasie pochodnej: $2 \cdot (x_1 + x_2) \dots$

```
n.cpp  TParam.h  TCandidate.h  TCandidate_Zad1.h  TPopulation.h  TAlgorithm.h
G2024  TCandidate  rate()
1  #pragma once
2  #include "TParam.h"
3
4  #define GENS_COUNT 2
5
6  class TCandidate
7  {
8  protected:
9      TParam genotype[GENS_COUNT] =
10     {
11         TParam("x1", 0, 100, 1),
12         TParam("x2", 0, 10, 1)
13     };
14
15     double mark;
16
17 public:
18     TCandidate();
19     TCandidate(const TCandidate &original);
20
21     double get_mark() const { return mark; };
22     virtual void rate();
23
24     void info();
25
26 protected:
27     void rand_gens_val();
28     double get_gen_val(int gen_id) const { return genotype[gen_id].get_val(); }
29 };
30
```

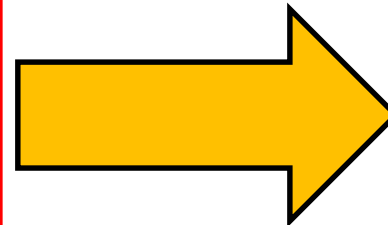
... w tym celu musimy zadeklarować funkcję *rate()* w klasie podstawowej jako **funkcję wirtualną**. Dzięki deklaracji wirtualności funkcji wskaźnik będzie sprawdzał na jakiej klasy obiekt wskazuje i wywoływał odpowiednią funkcję.



```
Konsola debugowania programu I
=====
== gens count: 2
== "x1" value: 71
== "x2" value: 9
==
== rate: 160
=====
```

Ponowne wykorzystanie wskaźnika

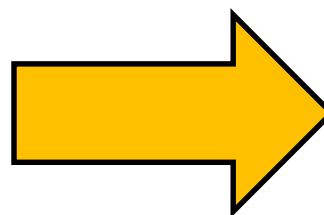
```
main.cpp  TParam.h  TCandidate.h  TCandidate_Zad1.h
AG2024 (Globalny zasięg)
1 #include <iostream>
2 #include <cstdlib> // srand
3 #include <time.h>
4
5 #include "TCandidate.h"
6 #include "TCandidate_Zad1.h"
7 #include "TCandidate_Zad2.h"
8
9 using namespace std;
10
11
12 int main()
13 {
14     srand(time(0));
15
16     TCandidate* os = new TCandidate_Zad1{};
17     os->rate();
18     os->info();
19     delete os;
20
21     os = new TCandidate_Zad2{};
22     os->rate();
23     os->info();
24     delete os;
25
26     std::cout << "\n\n\n";
27     return 0;
28 }
```



```
Konsola debugowania programu
=====
==
== gens count: 2
== "x1" value: 96
== "x2" value: 3
==
== rate: 198
=====

=====
==
== gens count: 2
== "x1" value: 6
== "x2" value: 19
==
== rate: 55
=====
```


Funkcja wirtualna - TEST



```
main.cpp  TParam.h  TCandidate.h  TCandidate_Zad1.h  TPop
AG2024 (Globalny zasięg)
1  #include <iostream>
2  #include <cstdlib> // srand
3  #include <time.h>
4  #include <vector>
5
6  #include "TCandidate.h"
7  #include "TCandidate_Zad1.h"
8  #include "TCandidate_Zad2.h"
9
10 using namespace std;
11
12
13 int main()
14 {
15     srand(time(0));
16
17     vector<TCandidate*> candidates;
18
19     candidates.push_back(new TCandidate{});
20     candidates.push_back(new TCandidate_Zad1{});
21     candidates.push_back(new TCandidate_Zad2{});
22
23     for (int i = 0; i < candidates.size(); i++)
24     {
25         candidates[i]->rate();
26         candidates[i]->info();
27     }
28
29     for (int i = 0; i < candidates.size(); i++)
30     {
31         delete candidates[i];
32     }
33
34     std::cout << "\n\n\n";
35     return 0;
36 }
```

```
Konsola debugowania programu
=====
==
== gens count: 2
== "x1" value: 95
== "x2" value: 6
==
== rate: 9031
=====

=====
==
== gens count: 2
== "x1" value: 82
== "x2" value: 4
==
== rate: 172
=====

=====
==
== gens count: 2
== "x1" value: 2
== "x2" value: 11
==
== rate: 15
=====
```

Tablica/wektor wskaźników pokazujących na obiekty różnych klas, związanych ze sobą związkami dziedziczenia.

Zadanie

Napisać kod umożliwiający wybór typu i liczby osobników zapisanych w wektorze.

```
Konsola debugowania programu Microsoft Visual Studio
Który osobnik [0-2]: 1
Ilu osobnikow utworzyc? 3

=====
==
== gens count: 2
== "x1" value: 29
== "x2" value: 9
==
== rate: 76
=====

=====
==
== gens count: 2
== "x1" value: 45
== "x2" value: 3
==
== rate: 96
=====

=====
==
== gens count: 2
== "x1" value: 9
== "x2" value: 0
==
== rate: 18
=====
```